

Você recebeu o material necessário para realizar a segunda avaliação. São quatro atividades práticas que devem ser desenvolvidas no computador. Enviar na atividade do Teams somente os quatro arquivos contendo os códigos fontes (**exercício01.py**, **exercício02.py**, **exercício03.py** e **exercício04.py**) . Não altere o nome do arquivo. A pontuação de cada atividade está na frente do enunciado. Manter os arquivos binários (*.bin) no mesmo diretório do código fonte.

Atividade 01:

Código: **exercício01.py**

(2 pontos) Implemente uma função chamada `extrair_dados` que recebe como parâmetro um dicionário no formato da resposta da API pública da série Rick and Morty. Essa resposta contém uma chave chamada "results", que armazena uma lista de personagens com diversas informações.

A função deve percorrer a lista de personagens e extrair apenas os seguintes campos de cada um:

- `id`: identificador do personagem,
- `nome`: nome do personagem,
- `especie`: espécie à qual o personagem pertence,
- `genero`: gênero do personagem.

Esses dados devem ser armazenados em um novo dicionário e adicionados a uma lista final. Ao final da execução, a função deve retornar essa lista com os dados extraídos.

Entrada:

```
entrada = {
  "results": [
    {
      "id": 1,
      "name": "Rick Sanchez",
      "status": "Alive",
      "species": "Human",
      "type": "",
      "gender": "Male"
    },
    {
      "id": 2,
      "name": "Morty Smith",
      "status": "Alive",
      "species": "Human",
      "type": "",
      "gender": "Male"
    }
  ]
}
```

Saída:

```
[
  {"id": 1, "nome": "Rick Sanchez", "especie": "Human", "genero": "Male"},
  {"id": 2, "nome": "Morty Smith", "especie": "Human", "genero": "Male"}
]
```

Atividade 02:

Código: **exercicio02.py**

(2 pontos)

A estrutura de dados apresentada é uma lista de dicionários, onde cada dicionário representa informações completas sobre um endereço consultado a partir de um CEP (Código de Endereçamento Postal), utilizando a API pública do ViaCEP.

Cada dicionário contém os seguintes campos:

- 'cep': o código postal formatado (ex: '01001-000');
- 'logradouro': nome da rua, avenida ou praça correspondente ao CEP;
- 'complemento': informações adicionais de localização (ex: "lado ímpar", "até o número X");
- 'unidade': campo geralmente em branco, utilizado para indicar unidade dos Correios (casos raros);
- 'bairro': o bairro em que o logradouro se localiza;
- 'localidade': o nome da cidade;
- 'uf': a sigla do estado (ex: 'SP' para São Paulo);
- 'estado': o nome completo do estado (campo adicionado ou enriquecido);
- 'regiao': a região geográfica do Brasil (ex: 'Sudeste');
- 'ibge': o código do município segundo o IBGE (Instituto Brasileiro de Geografia e Estatística);
- 'gia': código GIA (Guia de Informação e Apuração do ICMS), usado em SP;
- 'ddd': o código de Discagem Direta à Distância (ex: '11' para a capital paulista);
- 'siafi': código utilizado pelo governo federal para identificar municípios em sistemas administrativos.

Implemente uma função chamada `extrair_dados` que recebe como parâmetro uma lista de dicionários. Para cada item da lista, a função deve montar uma **tupla** com os três valores acima e adicioná-la a uma nova lista. Ao final, a função deve retornar essa lista de tuplas.

Entrada:

```
[
  {
    'cep': '01001-000',
    'logradouro': 'Praça da Sé',
    'complemento': 'lado ímpar',
    'unidade': '',
    'bairro': 'Sé',
    'localidade': 'São Paulo',
    'uf': 'SP',
    'estado': 'São Paulo',
    'regiao': 'Sudeste',
    'ibge': '3550308',
    'gia': '1004',
    'ddd': '11',
    'siafi': '7107'
  },
  {
```

```
'cep': '04003-010',
'logradouro': 'Rua Leônicio de Carvalho',
'complemento': '',
'unidade': '',
'bairro': 'Paraíso',
'localidade': 'São Paulo',
'uf': 'SP',
'estado': 'São Paulo',
'regiao': 'Sudeste',
'ibge': '3550308',
'gia': '1004',
'ddd': '11',
'siafi': '7107'
}
]
```

Saída:

```
[
('01001-000', 'São Paulo', 'São Paulo'),
('04003-010', 'São Paulo', 'São Paulo')
]
```

Atividade 03:

Código: **exercicio03.py**

O resultado armazenado na variável `dados` é uma lista de dicionários, onde cada dicionário representa os dados de um único dia de cotação.

Entrada:

```
[
  {
    "code": "USD",
    "codein": "BRL",
    "name": "Dólar Americano/Real Brasileiro",
    "high": "5.4734",
    "low": "5.4143",
    "varBid": "-0.0426",
    "pctChange": "-0.78",
    "bid": "5.4261",
    "ask": "5.4266",
    "timestamp": "1718994000",
    "create_date": "2024-06-21 21:00:01"
  },
  {
    "code": "USD",
    "codein": "BRL",
    "name": "Dólar Americano/Real Brasileiro",
    "high": "5.4502",
    "low": "5.4021",
    "varBid": "-0.0321",
    "pctChange": "-0.65",
    "bid": "5.4187",
    "ask": "5.4192",
    "timestamp": "1718907600",
    "create_date": "2024-06-20 21:00:01"
  }
]
```

Implemente as seguintes funções para realizar buscas na lista carregada em `dados`:

1.(2 pontos)

Desenvolva a função `search_high_values(dados)`. Esta função deve retornar uma lista com os valores do campo `high`.

Saída:

```
['5.4734', '5.4502']
```

2. (2 pontos)

Desenvolva a função `search_low_values(dados)`. Esta função deve retornar uma lista com os valores do campo `low`.

Saída:

```
['5.4143', '5.4021']
```

Atividade 04:

Código: **exercicio04.py**

Você recebeu um arquivo contendo dados consumidos de uma API do Star Wars.

Exemplo de Entrada:

```
[{'name': 'Luke Skywalker',
  'height': '172',
  'mass': '77',
  'hair_color': 'blond',
  'skin_color': 'fair',
  'eye_color': 'blue',
  'birth_year': '19BBY',
  'gender': 'male',
  'homeworld': 'https://swapi.info/api/planets/1',
  'films': ['https://swapi.info/api/films/1',
            'https://swapi.info/api/films/2',
            'https://swapi.info/api/films/3',
            'https://swapi.info/api/films/6'],
  'species': [],
  'vehicles': ['https://swapi.info/api/vehicles/14',
               'https://swapi.info/api/vehicles/30'],
  'starships': ['https://swapi.info/api/starships/12',
                 'https://swapi.info/api/starships/22'],
  'created': '2014-12-09T13:50:51.644000Z',
  'edited': '2014-12-20T21:17:56.891000Z',
  'url': 'https://swapi.info/api/people/1'},
 {'name': 'C-3PO',
  'height': '167',
  'mass': '75',
  'hair_color': 'n/a',
  'skin_color': 'gold',
  'eye_color': 'yellow',
  'birth_year': '112BBY',
  'gender': 'n/a',
  'homeworld': 'https://swapi.info/api/planets/1',
  'films': ['https://swapi.info/api/films/1',
            'https://swapi.info/api/films/2',
            'https://swapi.info/api/films/3',
            'https://swapi.info/api/films/4',
            'https://swapi.info/api/films/5',
            'https://swapi.info/api/films/6'],
  'species': ['https://swapi.info/api/species/2'],
  'vehicles': [],
  'starships': [],
  'created': '2014-12-10T15:10:51.357000Z',
  'edited': '2014-12-20T21:17:50.309000Z',
  'url': 'https://swapi.info/api/people/2'}]
```

(2 pontos) Implemente a função chamada **listar_nomes_personagens** que receba como parâmetro uma lista de dicionários de personagens e retorne o nome do personagem.

Saída:

['Luke Skywalker', 'C-3PO']